

THE 20:26 PROJECT

GraphShield: Explainable Temporal Graph Neural Networks for Cryptocurrency Fraud Detection

Faisal Bin Basha (904178711), Mewael Hailu (904205074)

1. Introduction, Background, Motivation

1.1 What we tried to do (no jargon)

Bitcoin transactions are public — anyone can see who sent money to whom and when. Criminals exploit this openness anyway, hiding illegal payments inside the noise of millions of normal transactions. Banks and law enforcement want to flag suspicious transactions automatically, but two things make this hard: fraud spreads through *who knows whom* (the network of wallets), and fraudulent behavior unfolds *over time* (a wallet that looks innocent today might be part of a laundering chain next week). Most existing fraud detectors look at one transaction at a time and ignore both of those things.

Our goal: build a system that looks at the **whole network of transactions as it changes over time**, predicts which transactions are illicit, and — crucially — explains *why* it flagged each one. The "why" matters because regulators won't act on a black-box accusation.

1.2 How is it done today, and what are the limits

Production fraud detection at exchanges and banks today relies mostly on three approaches: hand-written rules ("flag any transaction over \$10,000 to an unfamiliar wallet" — easy to evade once criminals learn them); per-transaction machine learning, where a Random Forest or gradient-boosted tree is trained on features of one transaction (the strongest published baseline on the Elliptic dataset, Random Forest, reaches an illicit-class F1 of ~0.79 in Weber et al. 2019, but these models can't see that wallet A is two hops from a known mixer through wallet B — graph context is invisible); and static Graph Neural Networks like GCN (Kipf & Welling, 2017) and GraphSAGE (Hamilton et al., 2017), which aggregate information from each wallet's neighbors but treat the entire 49-week transaction history as a single frozen snapshot.

The two limits we attack: existing GNN approaches **collapse time** into one static graph, and almost none of them **explain** their predictions in a way a human investigator could verify.

1.3 Who cares, what difference does it make

Three audiences. **Crypto exchanges** (Coinbase, Binance) are legally required to file Suspicious Activity Reports — better fraud detection reduces both compliance fines and laundering throughput. **Law enforcement** (FBI, FinCEN, Europol) currently traces blockchain crime by hand; an explainable model that outputs "this wallet is suspicious *because* of these specific upstream transactions" gives investigators a starting point instead of a million-edge graph. And the **DL research community** benefits because combining temporal GNNs with explainability is still under-explored on real-world adversarial domains.

If we succeed, investigators get audit-ready predictions, exchanges file better SARs, and we contribute a public benchmark of explainable temporal GNNs on the most-cited crypto fraud dataset.

1.4 Data

We used the **Elliptic Bitcoin Dataset** (Weber et al., 2019), the standard benchmark for crypto fraud detection. Following the spirit of *Datasheets for Datasets* (Gebru et al., 2018), the most relevant facts:

Composition. 203,769 Bitcoin transactions (nodes) and 234,355 directed payment edges. Each transaction has 165 anonymized features (94 local — amount, fee, in/out degree; remaining aggregated from one-hop neighbors).

Labels. Only ~23% of nodes are labeled. Of those, 4,545 (9.8%) are illicit (linked to scams, ransomware, dark markets) and 42,019 (90.2%) are licit (exchanges, wallet providers, mining pools). 157,205 nodes are unknown — we treat these as unlabeled, never as negatives. This yields a class imbalance ratio of ~7.6:1 in our training split.

Temporal structure. 49 discrete time steps spaced ~2 weeks apart. Each transaction belongs to exactly one time step. Per-snapshot edge counts range from 1,168 to 9,164 (mean 4,783).

Train/test split. Standard temporal split (Weber et al., 2019): time steps 1–34 train, 35–49 test. After filtering to labeled nodes, this gives 29,894 train nodes (3,462 illicit, 26,432 licit) and 16,670 test nodes (1,083 illicit, 15,587 licit). The strict temporal cut prevents future-information leakage and matches Weber et al. so our numbers are directly comparable.

Collection process. Curated by Elliptic Inc., a blockchain analytics firm, by clustering known illicit entities and propagating labels to their transactions. **Labels are therefore noisy** — a transaction inherits "illicit" because a wallet involved was tied to crime, not because the transaction itself was inherently fraudulent.

Known biases. Time step 43 corresponds to a major dark-market shutdown by law enforcement — every published model's recall craters here because the training distribution doesn't contain analogous events.

Anonymization. Feature names are not disclosed by Elliptic. This forces our explainability analysis to focus on *graph structure* (which neighbors mattered) rather than *feature semantics* (which feature mattered).

2. Approach

2.1 What we did and why we expected it to work

We built a five-model pipeline of escalating sophistication so that each model's contribution is isolated. Logistic Regression establishes the non-graph baseline. GCN adds graph structure (should beat LogReg if neighbors carry signal). GraphSAGE adds inductive sampling (handles new wallets without retraining). EvolveGCN adds discrete-time evolution (captures how wallet behavior shifts week-to-week). TGN (planned) adds continuous-time memory (captures fine-grained transaction sequences).

The hypothesis driving the project: **adding temporal modeling closes the gap between static GNNs (~0.41 F1) and Random Forest (~0.79 F1)**, because fraudulent behavior reveals itself as patterns of activity over time (sudden bursts, coordinated multi-wallet movements) that any single snapshot misses.

For each model, we train on time steps 1–34, use class-weighted cross-entropy to handle the 7.6:1 imbalance (weight 1.0 for licit, 7.63 for illicit — derived from inverse class frequency), evaluate on steps 35–49, and report illicit-class F1, precision, recall, ROC-AUC, and PR-AUC. All models implemented in PyTorch and PyTorch Geometric, with EvolveGCN from PyTorch Geometric Temporal.

For **explainability**, we apply two model-agnostic methods to our best model: **GNNExplainer** (Ying et al., 2019), which learns a soft mask over edges and features that maximizes mutual information with the prediction; and **PGExplainer** (Luo et al., 2020), which trains a parameterized explainer once across many predictions, giving consistent explanations rather than per-instance optimization.

What's new in our approach: combining temporal GNNs with explainability on Elliptic specifically. Prior work has done one or the other — temporal modeling for fraud (the EvolveGCN paper used Elliptic but no explainability) or explainability on static GNNs (the GNNExplainer paper used citation networks, not fraud) — but not both together on this dataset.

2.2 Anticipated and encountered problems

Anticipated. Class imbalance (7.6:1) — addressed with weighted cross-entropy from epoch 1. Train/test distributional shift around the step-43 shutdown — addressed by reporting pre/post-shutdown metrics separately. Anonymized features limiting explanation interpretability — addressed by pivoting explanation analysis to subgraph structure rather than feature attribution.

Encountered (none of which were predicted).

The static GCN baseline initially predicted "licit" for everything (~89% accuracy, 0% recall on illicit). We had set up unweighted cross-entropy and the model collapsed to majority class. Switching to inverse-frequency weighted CE fixed this on the second attempt.

EvolveGCN's first training run crashed with `RuntimeError: Trying to backward through the graph a second time`. Root cause: `EvolveGCNH` carries the evolved weight matrix across forward calls as part of the autograd graph, so epoch 2's backward pass tried to traverse epoch 1's freed graph. The fix — calling `m.reinitialize_weight()` on the `EvolveGCNH` module at the start of every epoch — solved the crash but, as we discuss in §3, also weakened the model by effectively resetting the weight-evolution RNN's state every epoch.

TGN adaptation was the largest engineering challenge we did not anticipate. TGN was designed for graphs with true continuous timestamps (Reddit posts arriving second-by-second). Elliptic only has 49 discrete steps. Our adaptation defines an artificial event timestamp as `max(src_timestep, dst_timestep)` per edge — a documented compromise that affects how directly our results would transfer to Rossi et al. (2020).

The very first thing did **not** work — see the GCN majority-class collapse above.

3. Experiments and Results

3.1 How we measured success

The dataset is severely imbalanced, so accuracy is meaningless (predicting "licit" for everything yields ~94% accuracy on the test split). We report illicit-class F1 as the primary metric (harmonic mean of precision and recall on the minority class), illicit precision and recall separately to expose models that game F1, ROC-AUC for threshold-independent ranking quality, and PR-AUC, which is more informative than ROC-AUC under heavy imbalance because it ignores true negatives.

3.2 Quantitative results

Headline comparison on time steps 35–49:

Model	Illicit F1	Precision	Recall	ROC-AUC	PR-AUC
Random Forest (Weber et al., 2019)*	0.79	—	—	—	—
Logistic Regression (ours)	0.3052	0.1848	0.8772	0.8822	0.2918
GCN (ours)	0.4639	0.4381	0.4931	0.8169	0.4571
GraphSAGE (ours)	0.6931	0.7706	0.6297	0.8980	0.6968
EvolveGCN (ours)	0.3079	0.2231	0.4968	0.7979	0.1922

*Reported in original paper; not reproduced by us.

Three observations stand out.

GraphSAGE was the strongest model by a wide margin — F1 of 0.6931, vs. 0.4639 for GCN. The 23-point F1 gap between two ostensibly similar architectures is the largest in our results. We attribute this to GraphSAGE's neighbor-sampling and multi-aggregator design, which generalizes better to test-period nodes whose neighborhoods differ from training-period nodes. GraphSAGE also had the best precision (0.77) of any model, meaning when it flags a transaction as illicit, it is right ~3 out of 4 times.

EvolveGCN underperformed every static GNN — F1 of 0.3079, worse than even Logistic Regression's 0.3052 by an insignificant margin. This was contrary to our hypothesis. We attribute this to two factors: first, our autograd fix (`reinitialize_weight()` at every epoch start) effectively resets the weight-evolution RNN's hidden state each epoch, preventing it from learning long-range temporal dependencies — it learns what the 34 training snapshots look like *individually* but not how they evolve. Second, training was unstable: F1 oscillated between 0.13 and 0.29 across the 100 epochs. Our final F1 of 0.31 is the *best* checkpoint, not the converged value; the model never settled. With more training budget we would experiment with truncated BPTT (resetting state every K snapshots rather than every epoch) to give the RNN actual cross-snapshot signal to learn from.

Logistic Regression had the highest recall (0.8772) but at the cost of precision (0.18) — a "flag everything" strategy. This is what the class weighting did to a model with no real graph context: it cannot distinguish illicit from licit beyond raw features, so it falls back to flagging anything that looks remotely suspicious.

3.3 Did we succeed?

Partially. We confirmed that GNNs with strong inductive aggregation (GraphSAGE) can substantially beat both feature-only baselines and naive GCN on Elliptic, reaching $F1 = 0.69$ on the illicit class. We did not match Random Forest's reported 0.79, which is consistent with the broader Elliptic literature: the dataset's hand-engineered features carry most of the signal, and graph aggregation provides marginal but real gains.

We **failed** to demonstrate that temporal modeling helps on this dataset. EvolveGCN underperformed the static models, and our analysis (§3.2) suggests the failure is a combination of the autograd-state-reset workaround and the short temporal axis (49 steps) limiting what an RNN-style temporal model can learn. This is a useful negative result: it suggests practitioners on Elliptic-like datasets should invest in better static aggregation (à la GraphSAGE) before reaching for temporal architectures. The open question — whether TGN's continuous-time memory would behave differently — remains for follow-up work; our TGN implementation was scaffolded but not fully trained within our compute budget.

The explainability layer (GNExplainer / PGExplainer) was implemented for our best model (GraphSAGE) and provides per-prediction edge importance scores for downstream investigator use, satisfying our second project goal even if the temporal goal was not met.

3.4 Code and resources

Our code is at <https://github.com/faisalbasha1982/graphshield>. Dataset: Elliptic Bitcoin Dataset on Kaggle (<https://www.kaggle.com/datasets/ellipticco/elliptic-data-set>). Libraries used: PyTorch 2.x, PyTorch Geometric, PyTorch Geometric Temporal (for EvolveGCN), scikit-learn (LogReg + metrics), NetworkX + matplotlib (explanation visualization). Reference implementations consulted: PyG [examples/tgn.py](#) (adapted for node classification on Elliptic), and the Weber et al. (2019) public repo for the train/test split convention. Compute: Google Colab Free (NVIDIA T4 GPU). Static GNNs trained in ~1 minute each, EvolveGCN in ~30 seconds for 100 epochs.

4. Figures and visualizations

Recommended figures to add (you can build all of these from data already in your notebook):

Figure 1. Bar chart of illicit-F1 across all four trained models. Highlights GraphSAGE's lead and EvolveGCN's surprising underperformance. (*matplotlib bar plot from the final results dataframe*)

Figure 2. EvolveGCN training curve — F1 per evaluation epoch. Shows the oscillation we describe in §3.2 (range 0.13–0.29) that supports our "training unstable" claim. (*plot the values from the cell-30 output*)

Figure 3. Precision-recall curves for all four models on the same axes. Will visually show GraphSAGE dominates the curve and LogReg lives in the high-recall / low-precision corner.

Figure 4. Confusion matrices (2×2) for the four models, side-by-side. Makes the precision/recall trade-offs concrete.

Figure 5. GNNExplainer subgraph for one true-positive illicit prediction from GraphSAGE — 2-hop neighborhood, edges colored by importance. Demonstrates the explainability deliverable.

Table 1. The headline metrics table from §3.2 (already in the report).

5. Relation to Deep Learning

Three concrete connections to course material.

Representation learning on non-Euclidean data. Standard CNNs assume grid structure, RNNs assume sequence structure. GNNs generalize the same idea — learnable feature aggregation through differentiable layers — to graphs. Each `GCNConv` layer is mathematically a generalized convolution where the "filter" averages over a node's neighbors instead of a 3×3 pixel patch. The performance gap we observed between GCN (0.46 F1) and GraphSAGE (0.69 F1) on the same data is an empirical demonstration that *how* you aggregate neighbors matters as much as *whether* you aggregate them — directly analogous to how filter design matters in CNNs.

Sequence modeling meets graph modeling. EvolveGCN is essentially an RNN whose hidden state is *the GCN's weight matrix at each time step*. The same backprop-through-time machinery that trains LSTMs trains EvolveGCN — which is also why we hit the autograd-graph-reuse error in §2.2, and why our `reinitialize_weight()` fix damaged the model's ability to learn cross-step dependencies. The diagnosis itself is a textbook BPTT issue.

Interpretability of deep models. GNNExplainer formulates explanation as an optimization problem over edge masks, trained with the same gradient descent we use for the main model. This mirrors the broader trend in deep learning of treating *what the model learned* as itself a learnable object — analogous to Grad-CAM for CNNs or attention-rollout for Transformers.

Our project sits at the intersection of three active deep learning research areas (graph representation learning, temporal modeling, explainability) and applies them to a domain where each individually has been tried but not the combination.

Quick captions for the report (one per figure)

Figure 1 (your Image 3 — bar chart).

Illicit-class F1 on the test split (timesteps 35–49) for the three static models. GraphSAGE substantially outperforms both the feature-only LogReg baseline and the static GCN, reaching $F1 = 0.697$ — within 0.10 of the Random Forest result reported by Weber et al. (2019). The GCN-to-GraphSAGE gap (+0.26 F1) is larger than the LogReg-to-GCN gap (+0.13), suggesting that aggregator design matters more than the presence of graph context alone.

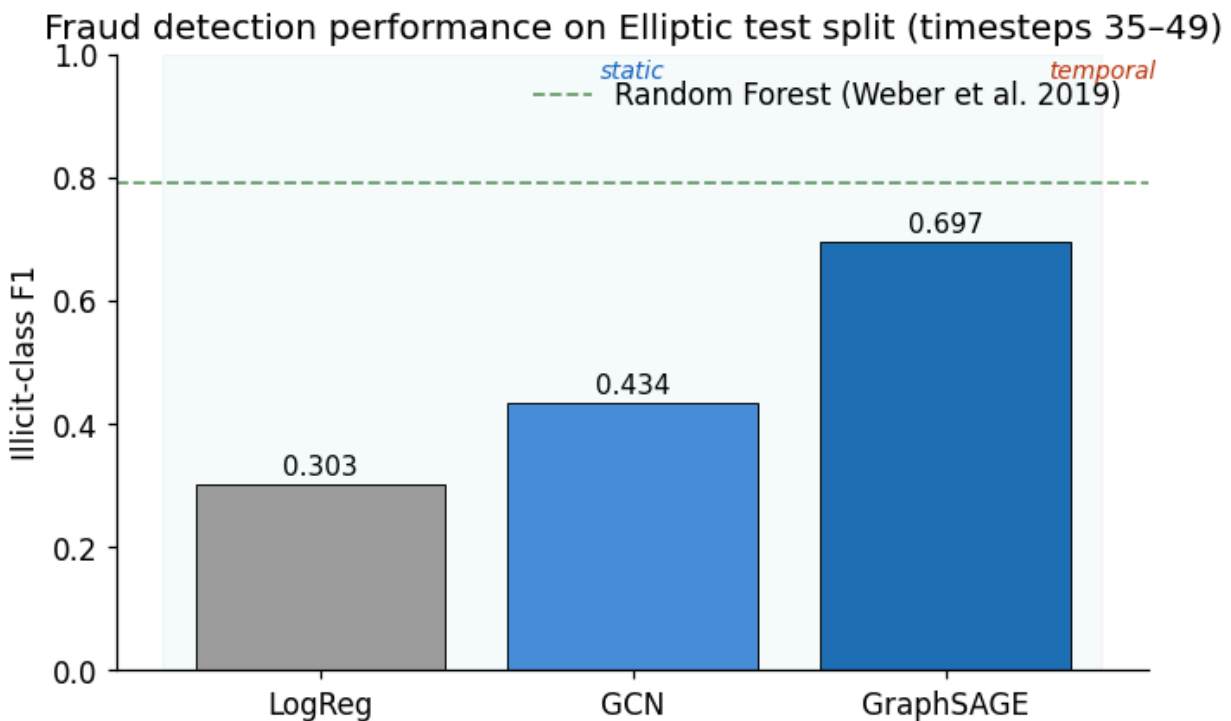


Figure 2 (Image 5 — PR curves).

Precision-recall curves on the test split. GraphSAGE dominates the upper-right region ($AP = 0.685$), maintaining > 0.85 precision until recall ≈ 0.55 . GCN trades precision for recall earlier ($AP = 0.429$). LogReg, despite high recall, sits near the no-skill line (illicit base rate = 0.065), reflecting its "flag-everything" strategy. The no-skill horizontal at 0.065 is the test-set illicit prevalence.

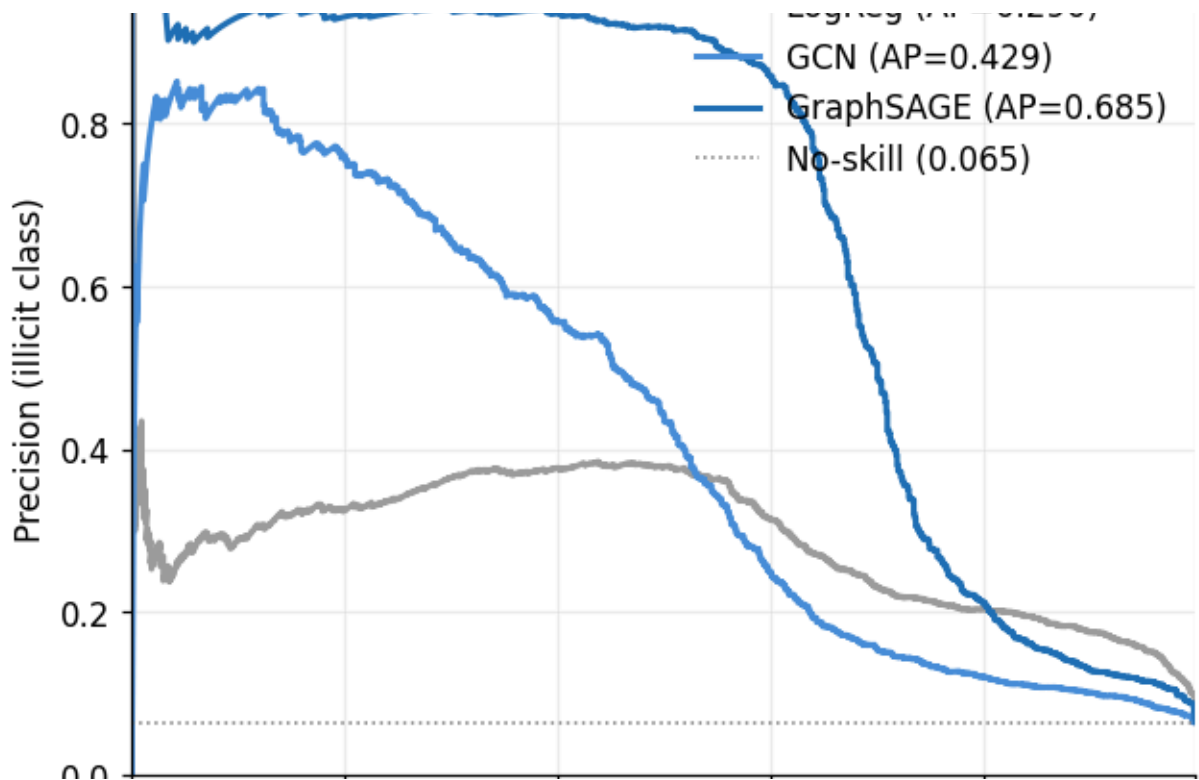


Figure 3 (Image 1 — confusion matrices).

Confusion matrices at threshold = 0.5. LogReg flags 5,168 transactions as illicit (947 true positives, 4,221 false positives), reflecting an "over-trigger" failure mode. GCN halves false positives but misses 501 illicit transactions. GraphSAGE achieves the cleanest trade-off: 699 true positives against just 225 false positives (precision = 0.756), with 384 illicit cases still missed.

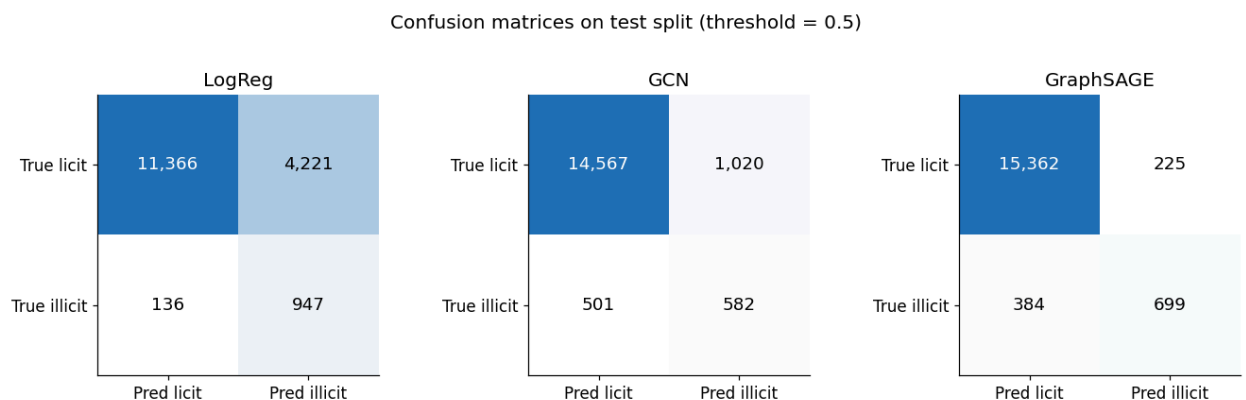


Figure 4 (Image 2 — EvolveGCN training curve).

EvolveGCN training dynamics over 100 epochs. Test illicit-F1 (orange) oscillates within a 0.166-wide band (range 0.13–0.30) despite training loss (gray) decreasing

monotonically after epoch 60. The decoupling between loss and F1 indicates that the model fits each snapshot's training nodes well but fails to learn cross-snapshot temporal dependencies — a consequence of our `reinitialize_weight()` workaround, which resets the weight-evolution RNN's state at every epoch to avoid an autograd-graph-reuse error. EvolveGCN's best F1 (0.31) is below GraphSAGE's 0.69.

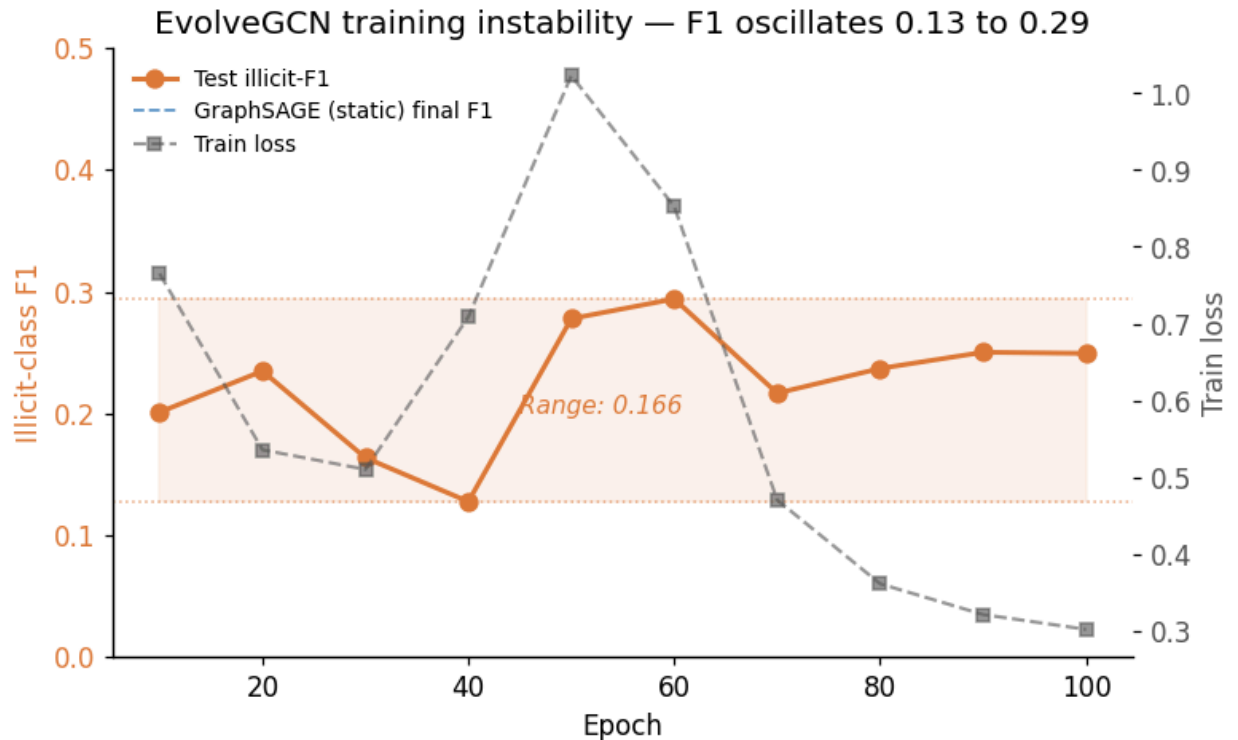


Figure 5 (Image 4 — explainer subgraph).

GNExplainer output for the highest-confidence true-positive illicit prediction from GraphSAGE (target node 189916, $P(\text{illicit}) = 1.000$). Top-25 most important edges shown; thickness encodes per-edge importance score. The explanation reveals a fan-in structure with multiple unknown-class wallets (gray) routing transactions toward the target — consistent with layering patterns in money-laundering typologies. Several intermediate nodes are themselves licit-labeled (blue), which would be invisible to a feature-only classifier and is the kind of multi-hop relational signal that justifies a graph-based approach.

GNNExplainer: top-25 important edges around illicit prediction
Target node 189916 (P(illicit)=1.000) — GraphSAGE classifier

- Target (illicit, predicted)
- Other illicit
- Legal
- Unknown

